

USING CLUSTERS FOR OPTIMAL ACCESS TO DATABASES

Course 11

1. The necessity of optimizing the access to DB
2. Clustering
3. Implementing the clusters for RDB
4. Trends for clusters' evolution

1. The necessity of optimizing the access to DB

The main reasons for optimizing the access to DB are:

- Obtaining a smaller amount of time spent with each request
- Storing space has to be as small as possible, by allocating it properly

The optimization is made during all the phases of developing a DB application: analysis, design, implementation.

Optimizing the time can be made by:

- Creating optimized requests that are based on relational calculus and algebra
- Using indexes
- Using clusters
- Using partitions

Optimizing the storing space can be made by:

- Reusing data
- Reallocating the space
- Compaction of data
- Dynamic allocation of space

2. Clustering

The most commonly definitions used for clusters (group of tables) are:

- ⊙ Physical grouping of data from one or more tables, which has the role of increasing the efficiency of data requests.
- ⊙ Method of compressed storage of data tables that have *common columns* and that are very often used together.

The key cluster is a column or group of columns that the tables have in common.

Clusters are created for tables that:

- are subjected to repeated requests and are rarely updated
- are frequently queried together
- participate in joins in multiple queries

Clusters have advantages for retrieving and disadvantages for updating.

The decision for using clusters is made considering the frequency of updates and the frequency of common queries on the clusters' tables.

Note: Clusters can have their own indexes.

Each value of the key cluster will be stored only once for each group of tables/**indexes**, no matter how often it is used in the group tables.

If the cluster and the associated index are created on different Tablespaces and stored on different disks then the speed of processing increases by accessing in parallel the 2 (n) disks.

Tablespace = a logical group of data files in a database. A DB can be stored in one ore many tablespaces. Oracle automatically create the SYSTEM tablespace. The tablespaces can be managed like folders (create, delete, merge).

A cluster index cannot have the UNIQUE constraint and cannot contain LONG columns.

Ex: Clusters are intensively used for documentary DB: libraries, archives, etc.

Clusters are not used for DB with intense transactions: hypermarkets, transportation, retail, telco, medical area, airports, etc.

3. Implementing the clusters for RDB

First, we create the cluster and then the tables in it.

- CREATE CLUSTER cluster_name (column1 NUMBER(5))
PCTFREE 10 PCTUSED 75
SIZE 600
TABLESPACE tbs_name
STORAGE (INITIAL 200K
NEXT 290K
MINEXTENTS 3
MAXEXTENTS 25
PCTINCREASE 30);
- CREATE TABLE table1 (column1 NUMBER(5) PRIMARY KEY, column2 NUMBER(10)) CLUSTER cluster_name(column1);
- CREATE TABLE table2 (column3 NUMBER(5) PRIMARY KEY, column4 NUMBER(10), column1 NUMBER(5)) CLUSTER cluster_name(column1);

Creating a cluster index means following these steps:

1. Create the cluster
2. Create the index
3. Insert data in tables

The command for creating an index on a cluster in Oracle is:

CREATE INDEX index_name ON CLUSTER cluster_name;

Designing the clusters is made accordingly to a physical level structure, the physical attributes being given only once for the group (PCTUSED, PCTFREE, SIZE, TABLESPACE, STORAGE).

For each database object the system has a physical data segment, that is composed of extents. Each extent has one or many data blocks.

The smallest physical structure in which Oracle stores data is called **data block** or **page**. It corresponds to a number of bytes of database stored on the disk. More data blocks that represent a certain type of information form an **extent**. The extents form together **segments** of data that are stored in the same *tablespace*. All data corresponding to a table is stored in a single *data segment* and all data corresponding to an index is stored in another segment, called *index segment*. If the table or index is *partitioned*, each partition is stored in its own segment.

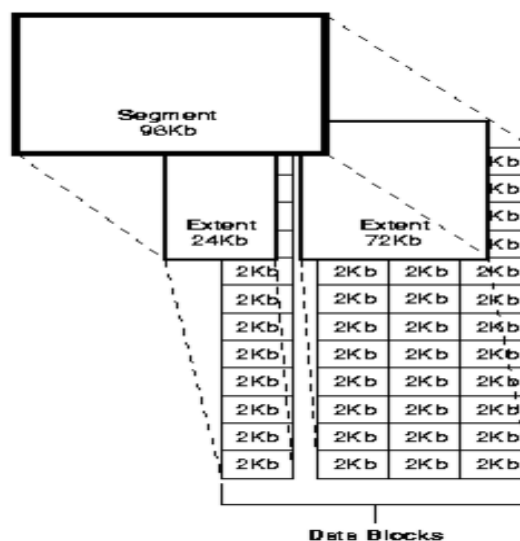


Fig. The structure of a data segment (source

https://docs.oracle.com/cd/B19306_01/server.102/b14220/logical.htm)

PCTUSED - specifies when the system (Oracle) should consider a data block to be empty enough to be added to the freelist (the list of data blocks that can receive new data).
PCTFREE - specifies how much space should be left in a data block for future updates.
SIZE – maximum size in bytes
INITIAL – initial size of the data segment
NEXT – size of the next extent
MINEXTENTS – minimum number of extents
MAXEXTENTS – maximum number of extents
PCTINCREASE - percentage by which each next extent (beginning with the third extend) will grow

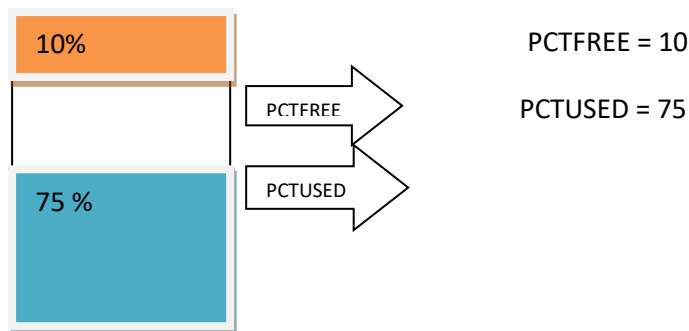


Fig. Data Block

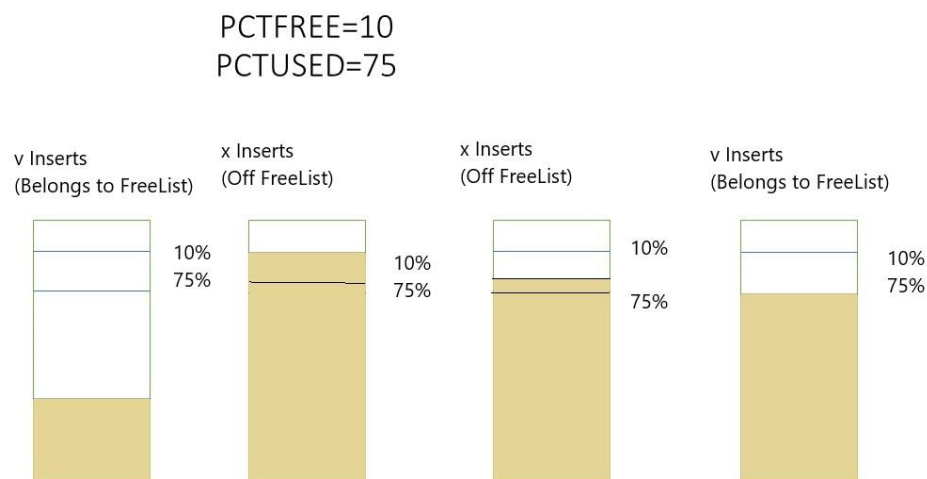


Fig. PCTFREE & PCTUSED

If PCTFREE is reached by the data stored in the block, new data is no longer inserted in this block until the PCTUSED limit is passed.
PCTUSED is the limit under which a data block has to have the occupied space in order to permit inserts again (if PCTFREE was previously reached).

Tablespaces and Datafiles

(https://docs.oracle.com/cd/B19306_01/server.102/b14220/physical.htm)

Oracle stores data logically in tablespaces and physically in datafiles associated with the corresponding tablespace. The next figure illustrates this relationship.

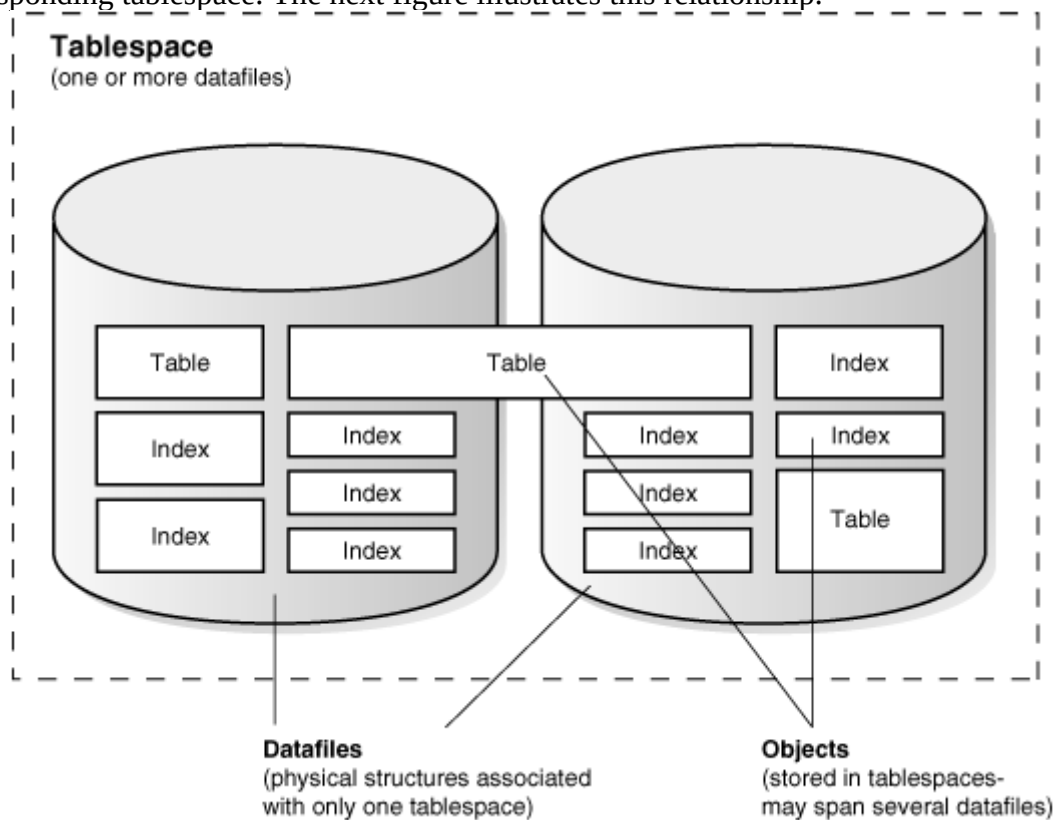


Fig. Datafiles and Tablespaces (source: Oracle Documentation)

Databases, tablespaces, and datafiles are closely related, but they have important differences:

- An Oracle database consists of one or more *logical storage units* called tablespaces, which collectively store all of the database's data.
- Each tablespace in an Oracle database consists of one or more files called datafiles, which are *physical structures* that conform to the operating system in which Oracle is running.
- A database's data is collectively stored in the datafiles that constitute each tablespace of the database. For example, the simplest Oracle database would have one tablespace and one datafile. Another database can have three tablespaces, each consisting of two datafiles (for a total of six datafiles).

For managing a cluster, the following commands are available in Oracle:

❶ ALTER CLUSTER cluster_name *option*

Ex: ALTER CLUSTER clust1

DEALLOCATE UNUSED KEEP 30 K;

❷ DROP CLUSTER cluster_name;

❸ DROP CLUSTER cluster_name INCLUDING TABLES;

❹ DROP CLUSTER cluster_name INCLUDING TABLES CASCADE CONSTRAINTS;

The following views offer information about the clusters:

- DBA_CLUSTERS – all clusters from the DB
- ALL_CLUSTERS – all clusters that are accessible to current user
- USER_CLUSTERS – all clusters in user's schema

Exercise: create a cluster for REGIONS and COUNTRIES tables based on *region id* clustering key.

Assume you already have Regions and Countries and you want to create new tables based on them, that follow the restrictions: *region_name* starts with A, *country_name* ends in a.

REGIONS: region_id, region_name

COUNTRIES: country_id, region_id, country_name

Solving the exercise:

```
create cluster clust1(region_id NUMBER);
CREATE INDEX idx_cluster ON CLUSTER clust1;
create table regions_cluster cluster clust1(region_id) as select * from regions
where region_name like 'A%';
create table countries_cluster cluster clust1(region_id) as select * from countries
where country_name like '%a';
select * from user_clusters;
--DROP CLUSTER clust1 INCLUDING TABLES;
```

4. Trends for clusters' evolution

DBMS that are dedicated to clustering for distributed DB:

- Oracle Database RAC – Real Application Cluster
- Sybase Adaptive Server Enterprise Cluster Edition
- IBM DB2 pureScale

RAC allows to have on different computers (devices) applications that access the same DB, thus offering clustering.

It has a share-everything architecture, meaning it shares the disk space and memory in a cluster.

The evolution trends that began with clustering can be seen in figure 8.3.

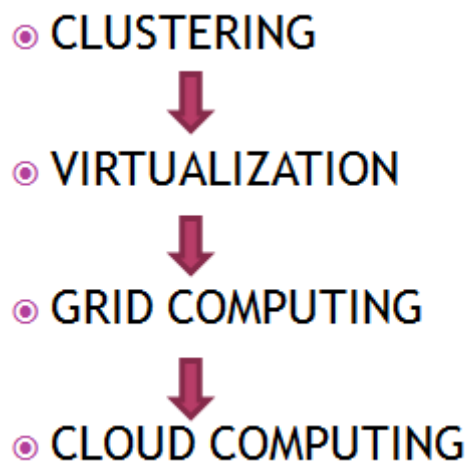


Fig. Clustering evolution

VIRTUALIZATION assumes the hardware resources are extended (to a virtual level) for a better sharing of the resources between application and users.

Applications <-> Virtualization <-> Storage

Ex. Virtual Machines

GRID COMPUTING is a particular Distributed Computing in which resources are put together in order to reach a goal.

It is used for applications with fewer queries.

CLOUD COMPUTING means the use of (hardware and software) resources that are delivered as services through a network (Internet).

Difference between



CLOUD COMPUTING

&

GRID COMPUTING



<https://www.geeksforgeeks.org/difference-between-cloud-computing-and-grid-computing/>